Hoja de Proyecto: Módulo de Calificación para Programación a Bajo Nivel

Descripción del Proyecto

El objetivo es desarrollar el módulo principal de corrección automática para una prueba de certificación en desarrollo de software a bajo nivel para videoconsolas portátiles. El sistema tomará una plantilla de respuestas correctas y las contrastará con las opciones elegidas por el candidato, calculando la nota final y determinando si existen anomalías en la prueba.

Lógica de Puntuación

El algoritmo de calificación debe aplicar un criterio de evaluación estricto para medir el conocimiento real:

- Las respuestas correctas suman puntos al total.
- Las respuestas incorrectas deben penalizar restando puntos al total para desincentivar las elecciones al azar.
- Las respuestas omitidas (en blanco) se ignoran en el cálculo; no deben sumar recompensas artificiales de puntos ni tampoco restar.

Especificaciones Técnicas y Requisitos

1. Estructura de Datos Básica

- Declara dos arreglos (arrays) de tipo entero de tamaño fijo en el bloque de la función principal.
- El primer arreglo contendrá la clave de respuestas correctas (valores del 1 al 4) y el segundo las respuestas del candidato, donde el valor 0 representa una pregunta omitida.
- Debido a las restricciones del nivel actual, los arreglos no se pasarán a funciones externas, sino que se iterarán localmente.

2. Control de Flujo y Categorización

- Utiliza un bucle `for` para iterar de manera secuencial a través de los índices de ambos arreglos simultáneamente.
- Implementa una estructura de control `switch` (o condicionales `if-else` en cascada) dentro del bucle para categorizar cada respuesta como acierto, error u omisión, incrementando los contadores correspondientes.

3. Diseño de Funciones y Prototipos

- Define una función independiente encargada de procesar los contadores y calcular la nota final.
- El prototipo de esta función debe declararse explícitamente antes de `main()`.
- Si la función no devuelve un valor mediante la palabra clave `return`, debe utilizar explícitamente el tipo `void` en su firma.
- Si la función no recibe parámetros convencionales por valor en algún espacio, debe especificar `(void)` en su lista de parámetros.

4. Manipulación de Memoria mediante Punteros

- La función de cálculo recibirá los resultados a través del paso por referencia utilizando punteros.
- Debe aceptar al menos un puntero a un tipo `float` (para la nota final) y un puntero a un tipo `int` (para una bandera booleana de anomalías).
- Dentro de la función, emplea el operador de indirección o desreferenciación (`*`) para modificar directamente las variables que residen en el ámbito de la función llamadora.

5. Seguridad e Inicialización

- Declara las variables puntero necesarias e inicialízalas inmediatamente con el valor especial `NULL` para prevenir referencias a basura en la memoria.
- Posteriormente, reasigna estos punteros utilizando el operador de dirección (`&`) para que apunten a las variables locales destinadas a almacenar los resultados finales.

6. Evaluación Final

- Emplea el operador ternario (`?:`) como expresión final para determinar el texto a mostrar en pantalla.

- Si la calificación final almacenada es mayor o igual a 5.0, debe evaluar a la cadena "Aprobado"; en caso contrario, a "Suspenso".